

전연욱

Game Client Programmer

“모두에게 추억으로 남을 게임을 개발합니다.”

유저에게 추억을 남기는 것을 넘어, 함께 프로젝트를 진행하는 팀원에게 추억이 될 경험을 만들어가는 것을 목표로 합니다.

정돈된 코드와 구조는 일하는 동안 스트레스를 줄이고 새로운 세상을 만들어가는 데에 집중할 수 있게 해줍니다.

또한 자연스럽게 최적화를 이끌어내며 UX를 향상시킬 수도 있습니다.

그렇기에 저는 모두에게 추억이 될 수 있도록, 기본적인 설계 구조와 컨벤션을 준수하며, 분석한 트렌드를 바탕으로 세계를 만들어갑니다.

학력

중앙대학교 졸업
(산업 보안 / 사이버 보안)

자격

정보처리 산업기사
(2019.11)

라이프 디자이너 1급
(2023.03)

교육

삼성 청년 SW-AI 아카데미 수료
(2025.07 ~ 2026.06)

대외 활동

신한 DS 미래 위협 협의회 (중앙대)
(2018.05 ~ 2018.12)

기술 스택

프로젝트 경험을 통해 다져온 기술과 협업 역량입니다.



C#

중 ● ● ● ● ●

1. OCP, Singleton 등 개발 패턴 적용
2. 기능 특성에 맞춘 개발 방법론 적용



파이썬

중하 ● ● ● ● ●

1. Django REST API 서버 구축



유니티

중 ● ● ● ● ●

1. 비대칭 PVP, 디펜스 등 다양한 장르의 게임 프로젝트 진행
2. 배포 플랫폼 특성에 맞춘 최적화 진행 (Windows, WebGL)



알고리즘

중 ● ● ● ● ●

1. Min-Heap 활용, 보스 패턴 디자인
2. Bit 연산 활용, DB 칼럼 최적화
3. DFS backtracking 활용, 미로 생성 기능 구현



커뮤니케이션

중상 ● ● ● ● ●

1. 다양한 직군 간 원활한 의사소통을 위한 문서화 체계 구축
2. 퍼실리테이터 역량 활용, 협업 프로세스 개선 및 소통 지원

개발 프로젝트

세 가지 게임 프로젝트를 통해 클라이언트 개발 경험을 쌓았습니다.



01 2D RPG · Web Game

프라가라흐

<2D RPG Web 게임>

2025-09-30 ~ 12-25 · 2인
Phaser · Django · Vue

USP

손쉬운 스킬 변경과 5가지 독창적인 스탯을 기반으로
나만의 빌드를 경험할 수 있는 전통 RPG



02 Horror PVP · Windows

마네킹: 헌티드 몰

<비대칭 PVP>

2026-02-20 ~ 03-25 · 6인
Unity · Photon Fusion2

USP

증강 시스템을 통해, 게임의 템포를 가속하는
공포 비대칭 PVP 게임



03 Defense · Web Game

헤이터

<디펜스 Web 게임>

2026-04-22 ~ 05-21 · 6인
Unity · Spring · React

USP

현재와 미래를 저울질 하는 디펜스 게임

01

PROJECT

SCOPE

- 보스 몬스터 설계
 - Min Heap 자료구조 활용
- 컷씬 시스템 설계
 - Bit 연산 활용
- 시스템 최적화 및 버그 대응
- API 서버 구축
- Web 배포

2D RPG · Web Game

프라가라흐

Phaser 엔진 기반 2D RPG Web 게임



PERIOD

2025.09.30 ~ 12.25

TEAM

2명

STACK

Phaser · Django · Vue

Django · SQLite3 기반 REST-API 서버 구축

게임 로직과 데이터 분리를 통한 런타임 패치 구조 설계

목적

- 게임 로직과 데이터 분리

구조 이점

- API 서버에서 데이터를 전달받는 구조
→ 밸런스 패치 시, 클라이언트 빌드 없이 DB 수치 조정만으로 패치 가능

발견된 한계

- SQLite3 의 write-lock 한계 인지

한계 수용 판단 근거

- 대다수 API 요청은 '조회'에 치중
- '쓰기' 요청은 회원가입·저장 시점에만 발생
- 배포 대상: 삼성 청년 SW-AI 아카데미 서울 캠퍼스 14기(약 1,000명)로 제한



클라이언트 시스템 설계 : Min-Heap 기반 보스 패턴 설계

Heap 자료구조를 통한 균형 잡힌 보스 패턴 설계

WHY

문제 — 기존 방식의 한계

X

랜덤 함수

플레이어의 운에 따라 강력한/약한 패턴이 반복 발생

X

쿨타임 기반

쿨타임 설계에 따라 특정 시점에 패턴이 몰아치는 경우 발생

✓

우선순위 기반 Min-Heap

패턴이 균형 있게 혼합, $O(\log N)$ 삽입으로 추가 발생 시간 최소화

Open-Close Principle 기반 리팩토링

- Before : switch문 → 새 패턴 추가 시, switch문 수정
- After : Dictionary → 새 패턴 추가 시, 항목 추가

HOW

구현 — Dictionary + Min-Heap

```
function CastSkill(skill, scene) {
  if (BossInstance.name == 'coffin') {
    switch (skill) {
      case 999:
        patternSet['thunder'].tryCast(scene, BossInstance);
        cooltime(scene, 999, 2.5);
        break;
      /* 이후 생략 */
    }
  }
}
```



```
const BossPatterns = {
  coffin: {
    999: { key:'thunder', cool:2.5, init:1 },
    1: { key:'summons', cool:90, init:5 },
    2: { key:'fireshoot', cool:30, init:10 },
    /* 이후 생략 */
  };
};
```

클라이언트 시스템 설계 : 컷씬 확인 시스템

Scene Number 비트 플래그를 통한 컷씬 시청 여부 관리

비트 연산 기반 체크 시스템

- Scene Number 기반 컷씬 시청 여부 확인
- DB에 int 형으로 저장되어 리소스 절약
- 빠른 속도의 비트 연산

```
if ((this.playerStats.cutScene & (1 << this.sceneNumber)) == 0) {
    this.cutscene.play(introScript);
    this.playerStats.cutScene += 1 << this.sceneNumber;
} else {
    this.cutsceneLock = false;
}
```

// IN-GAME Image



비정형 AI 코드 역분석 및 리팩토링

바이브 코딩으로 생성된 높은 Coupling 기능의 버그 분석 및 해결

CONTEXT

별도의 컨벤션·컨텍스트 설정 없이 진행된 팀원의 바이브 코딩 · AI가 생성한 막대한 양의 코드가 높은 Coupling 기반으로 동작

// CASE A

스킬 시스템 로직

버그 상황

- 스킬 포인트 투자 시, 특정 스킬에만 변경 사항이 적용

버그 분석

- 일부 스킬의 로직이 Scene에 바인딩
- 일부 스킬의 Override 과정에서 base 누락

해결 방법

- 각 스킬 코드의 흐름도 및 주석을 동봉한 리팩토링 문서를 작성하여 팀원에게 인계
- 팀원의 일을 대신 하기보다 조언으로 상호 성장이 이루어질 때, 더 긍정적인 결과로 이어질 것이라 판단

// CASE B

데이터 동기화

버그 상황

- 맵 이동·재로그인 이후 해금한 스킬이 미해금 상태로 변경

버그 분석

- API 오류를 대비한 더미 데이터에서 ‘레벨’ 데이터의 갱신 누락

해결 방법

- API의 개별 응답 데이터를 user 데이터로 통합한 구조체로 전환
- 클라이언트의 더미 데이터도 수정된 응답에 맞춰 구조체로 전환

02

PROJECT

SCOPE

- 살인마 진영 구현
- 생존자 진영 구현
- 상호작용 시스템 설계
- 증강 시스템 설계
- 멀티 플레이 시스템 설계
- FX 디자인
- 레벨(맵) 디자인
- UI/UX 디자인
- 시스템 최적화

Horror PVP · Windows

마네킹 : 헌티드 몰

Unity 엔진 기반 비대칭 PVP 게임



PERIOD

2026.02.20 ~ 03.25

TEAM

6명

STACK

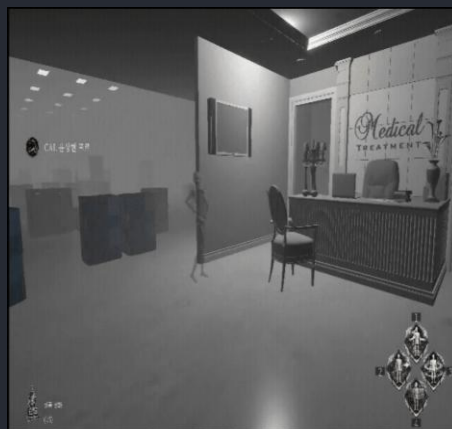
Unity · Photon Fusion2

살인마 진영 구현

설치 · 전이 · 교란 · 위장

설치

- 전이·교란에 사용되는 더미 설치
- 리스트 순회 vs Physics Overlap 비교
 - 총 더미는 15개 적은 수
 - Overlap의 경우, 오버헤드 대비 효율 낮음
 - 리스트 순회 채택



전이

- 선택한 더미와 위치 변경
- 빠른 속도의 맵 종횡 및 추적 요소로 사용
- Culling Mask를 통해, 투시 시점 구현
- 전용 셰이더로 안개의 영향 제거



교란

- 전이 스킬 사용 시 동작하는 애니메이션을 모든 더미에서 재생
- 전이·교란의 심리전 요소로 사용



위장

- 3 인칭 시점으로 변경, 살인마 SE 재생 정지
- 부족한 색적 능력 보완
- 더미로 위장하여 주요 지점에서 생존자를 유도



인터페이스 기반 상호작용 시스템 설계

인터페이스를 통한 상호작용 기능 간의 Coupling 감소

WHY

왜 인터페이스인가

- C#의 단일 상속 제약으로 추상 클래스로 일원화 불가
- 파괴 / 저주 / 뛰어넘기 → 수평적 확장 관계
- 파괴 / 저주 : 모든 유저가 공유해야 할 상태 → NetworkBehaviour 필요
- 뛰어넘기 : 위치값 동기화만 필요 → MonoBehaviour로 충분

```
public interface IInteractable
{
    string GetInteractionText(GameObject interactor);
    string GetAnimationTrigger();
    void OnInteract(GameObject gameObject);
}
```



추상 클래스 기반 증강 시스템 설계

추상 함수와 가상 함수를 통한 컴파일 안전성 및 기능 확장 구조

WHY

- 모든 유저가 증강 활성화 유무를 공유 → NetworkBehaviour 필요
- 기능이 큰 틀에서 수직적으로 확장
- Activate() : 실질 동작은 필수 → 추상 함수
- Awake() : 기본 동작 동일 + 확장 가능 → 가상 함수

```
public abstract class AbilityBase : NetworkBehaviour
{
    public abstract void Activate();

    protected virtual void Awake()
    {
        this.enabled = false;
    }
}
```



기능 최적화 : 스폰 시스템 최적화

데이터 압축을 통한 전송 데이터 최소화

BEFORE

기존의 스폰 시스템

- 4개 스폰 스팟, 6인 플레이어
- 플레이어 당 position(x,y,z) + rotation(x,y,z) → 6개 데이터 전송
- 고정값을 사용하는 position.x · rotation.x · rotation.z

```
public (Vector3 pos, Quaternion rot) GetKillerSpawnData()
{
    Vector3 data = killerPoints[SelectedIndex];
    Vector3 pos = new Vector3(data.x, 108f, data.z);
    Quaternion rot = Quaternion.Euler(0, data.y, 0);

    return (pos, rot);
}

public void PlayerJoined(PlayerRef player)
{
    /* 일부 생략 */
    var data = SpawnManager.Instance.GetKillerSpawnData();
    spawnPos = data.position;
    spawnRot = data.rotation;

    Runner.Spawn(prefabToSpawn, spawnPos, spawnRot, player);
}
```



전송 데이터 50% 감소

AFTER

최적화된 스폰 시스템

- 고정값(pos.x · rot.x · rot.z) 제거, 가변값만 Vector3 한 덩어리로 전송
- 플레이어 당 전송 데이터를 절반으로 압축

```
public (Vector3 dataSet) GetKillerSpawnData()
{
    Vector3 data = killerPoints[SelectedIndex];

    return (data);
}

public void PlayerJoined(PlayerRef player)
{
    /* 일부 생략 */
    var dataSet = SpawnManager.Instance.GetKillerSpawnData();

    Runner.Spawn(prefabToSpawn, dataSet, player);
}
```

기능 최적화 : Occlusion Culling

Occlusion Culling을 통한 AI 생성 오브젝트로 인한 프레임 드랍 해소

CONTEXT

배경 상황

- AI로 생성한 소도구 3D Mesh를 다수 사용
- scale 설정 이슈로 맵을 포함하여 모든 오브젝트가 100배로 확대
- Frustum Culling만으로는 일부 지역의 프레임 드랍 해소 불가

PROCESS

최적화 과정

1. 주요 프레임 드랍 지역에서 'AI 소도구', '일반 소도구', '구조물'을 하나씩 제거하며 원인 추적
→ AI로 생성한 소도구 제거 시, 드랍 현상이 가장 높은 쪽으로 해소됨을 확인
2. AI로 생성한 3D Mesh의 Vertex가 필요 이상으로 복잡함을 확인
→ LOD 적용 검토
3. LOD 부분 적용 결과, 효과 미미 + 짧은 개발 기간에 모든 오브젝트 적용 불가 판단
4. Occlusion Culling 적용
→ 가려진 소도구를 렌더링하지 않도록 처리하여 프레임 드랍 현상 최소화

BEFORE

Graphics:

13.0 FPS (77.2ms)

CPU: main 77.2ms render thread 47.9ms

Batches: 24492 Saved by batching: 0

300% 성능 향상



AFTER

Graphics:

63.6 FPS (15.7ms)

CPU: main 15.7ms render thread 9.4ms

Batches: 1807 Saved by batching: 3822

기능 최적화 : Jitter 해결

서버·클라이언트 상태 동기화 일원화

CONTEXT

배경 상황

- 특정 증강 획득 시 Jitter 발생
- 서버 : Tick 단위 계산 수행
- 클라이언트 : Frame 단위 계산 수행

ANALYSIS

원인 분석

- 서로 다른 시간 단위로 상태 값을 개별 수정
→ 네트워크 지연 발생 시, Jitter 극심

SOLUTION

해결 과정

01

동기화 일원화

증강 획득 시 상태 동기화 작업을 서버로 일원화

↓

02

RPC 확정

이후 RPC를 통해 클라이언트의 상태 값을 한 번 더 확정

↓

03

적용 시점 지연

증강의 적용 시점을 동기화 확인 이후로 의도적으로 지연

03

PROJECT

SCOPE

- 시스템 매니저 설계
- 디펜스 기능 구현
- 아군/적 유닛 설계
- 정찰/탐사 이벤트 구현
(15 시나리오 중 4개 시나리오 구현)
- FX 디자인
- UI/UX 디자인
- 시스템 최적화
- API 서버 구축
- 서버 내 전투 검증 시스템 구축
- 통신 암호화
- Web 배포
- 데이터 로그 분석 (밸런스 패치)

Defense · Web Game

헤이터

Unity 엔진 기반 디펜스 Web 게임



PERIOD

2026.04.22 ~ 05.21

TEAM

6명

STACK

Unity · Spring · React

초기 기획 총괄

초기 기획 총괄 및 담당자의 기획 수정 내용 조율

초기 기획	수정 기획	비고
탐사 (단일 콘텐츠)	정찰 / 탐사 (콘텐츠 이원화)	기존의 탐사 콘텐츠는 Low Risk – Low Return의 정찰 콘텐츠로 변경 High Risk – High Return의 신규 탐사 콘텐츠 추가 탐사 콘텐츠는 15종의 시나리오를 기반으로 세계관 몰입도 강화
사령관	효과 적용 방식 변경 (Cell 배치 → 패시브)	Cell을 소모하여 인근 유닛에 패시브 효과를 주는 방식 → 전체 유닛에게 패시브 효과 제공
아군 유닛	신규 병과 추가 (토텐 : Cell 배치 패시브 유닛)	초기 기획의 사령관 기능 계승
적 유닛	특성 삭제	일정 및 가용 에셋의 한계로 삭제 스탯으로 각 유닛의 유불리 구현
-	영지 콘텐츠 추가	추가 재화 및 특수 효과를 얻을 수 있는 영지 강화 시스템 추가
-	소모 재화 추가	전략 자원을 추가함으로써 전술적 고민 확장

시스템 설계 : Thread-safe Singleton 설계

제네릭 베이스 클래스 기반 Singleton을 통한 타입 안전성과 멀티스레드 안정성 확보

DESIGN

제네릭 베이스 클래스 기반 Singleton 패턴

- 각 매니저 객체에서 Singleton 코드를 제거하여 유지보수성 확보
- CRTP 패턴의 제네릭 베이스 클래스로 타입 안전성 확보
- 멀티스레드 환경 대비 상호 배제 추가

PROBLEM

Inspector 계층 정리

- Inspector 계층 정리를 위해 하나의 부모 오브젝트의 자식으로 매니저를 정리
- Unity 내부 제약으로 DontDestroyOnLoad() 미동작

SOLUTION

해결 방법 결정 : 부모 오브젝트 제거

- Singleton 컴포넌트 위치 변경 (매니저 → 부모)
→ 캐싱으로 수동 구현, 타입 안전성 상실
- 부모 오브젝트 제거 방식 채택

```
public class Singleton<T> : MonoBehaviour
    where T : MonoBehaviour
{
    private static T _instance;
    private static readonly object _lock = new object();

    public static T Instance
    {
        get
        {
            lock (_lock)
            {
                _instance = Object.FindAnyObjectByType<T>();
                if (_instance != null)
                {
                    MonoBehaviour mb = _instance as MonoBehaviour;
                    if (mb.transform.parent != null)
                        mb.transform.SetParent(null);
                    /* 싱글톤 생성 */
                }
            }
        }
    }
}
```

시스템 설계 : 게임 세션 복원 시스템

OCP 기반 세션 상태 복원 시스템

PROBLEM

- 새로그침 시, Scene 재로드
- 상태 미저장 시, 다음 스테이지 정보를 가진 채 플레이하거나 특정 콘텐츠를 무한 반복 가능

BEFORE

Switch 구조

- 각 상태를 switch문으로 분기 처리
→ OCP 위반

AFTER

Dictionary<상태, 핸들러> 매핑 구조

- OCP 준수, 신규 상태 추가 시 핸들러만 등록
- 상태별 복원 로직 분리로 유지보수성 향상
- 게임 세션 복원 시 핸들러 매핑만 조회

```
void Awake()
{
    Action recover = () => StartCoroutine(
        RecoveryTransitionRoutine("Defense"));

    _handlers = new Dictionary<GameState, Action>
    {
        { GameState.Preparation, recover },
        { GameState.Battle,      recover },
        { GameState.PostStage,   LoadPostStage },
    };
}

public void CheckAndRecoverySession()
{
    var state = GetLastSavedState();
    if (_handlers.TryGetValue(state, out Action handler))
        handler();
}
```

디펜스 기능 구현 : 유닛 배치 시스템

UI ↔ 월드 간 단방향 동기화를 통한 생성/파괴 최소화
사정거리 인디케이터를 통한 전략적 배치

GENERATION

오브젝트 생성/파괴 최소화

- UI 배치 종료 후 한 번에 월드에 단방향 동기화
- 생성 시 고유 ID를 부여하여 '배치 이동 유닛'은 위치 변경
→ 오브젝트 생성/파괴 최소화
→ 버프·HP 등 유닛의 상태 보존

RANGE INDICATOR

사정거리 인디케이터

- 전략적 유닛 배치를 위해 사정거리 가시화
- 버프 등으로 사정거리 증가 시, 별도 색으로 표현
- 고정값이 아니라 가장 짧은 사정거리(방패병) 기준 계산
→ 밸런스 패치 시 클라이언트 수정 최소화

```
/* 유닛 배치 */
public void SyncDeploymentWithWorld()
{
    var current = UIGrid.GetCurrentDeploymentWithIds();

    /* 제거된 유닛: 배치 취소 */
    var toRemove = _activeUnits.Keys
        .Where(n => !current.ContainsKey(n)).ToList();
    foreach (var n in toRemove)
        StartCoroutine(RetreatAndDestroy(_activeUnits[n]));

    /* 신규/이동 유닛 처리 */
    foreach (var item in current)
    {
        if (_activeUnits.TryGetValue(item.Key, out var existing))
            StartCoroutine(TransitionUnit(existing));
        else
            StartCoroutine(DeployUnit(item));
    }
}

/* 사정거리 인디케이터 */
float baseScale = (unitRange / shieldRange) * 1.4f;
float effectiveRange = unitRange * (1f + percentSum) + flatSum;
float effectiveScale = (effectiveRange / shieldRange) * 1.4f;
```



디펜스 기능 구현 : 오브젝트 풀 기반 스폰 시스템

오브젝트 풀, Pending Queue를 통한 대규모 유닛의 Garbage Collection 스파이크 방지

CONTEXT

배경 상황

- Web 게임 : 브라우저 리소스 공유 → Garbage Collection 비용에 민감
- 컨셉상 대량의 적 유닛 등장(최소 300 ~ 최대 1500)
→ 모든 유닛 생성/파괴 시 Garbage Collection 스파이크 발생 가능

SOLUTION

오브젝트 풀 + Pending Queue

- 웨이브 유닛 중 40%의 사전 풀 생성
- 풀 고갈 시 Pending Queue에 적재
- 유닛 반납 이벤트 발생 시 Pending 처리
→ Garbage Collection 스파이크 방지 + 메모리 안정성 확보

```
int poolSize = Mathf.CeilToInt(enemyData.count * poolFillRatio);

private void SpawnEnemy(int unitId, int grade)
{
    // Get() 실패 시 pending 증가
    GameObject obj = enemyObjectPool.Get(unitId);
    if (obj == null)
    {
        _pendingSpawn[unitId]++;
        return;
    }

    ApplyGrade(obj, unitId, grade);
}

// 적 유닛 반납 시 pending 처리
private void OnEnemyReturnedHandler(int unitId)
{
    if (_pendingSpawn[unitId] <= 0) return;

    _pendingSpawn[unitId]--;
    SpawnEnemy(unitId, _unitGrades[unitId]);
}
```

암호화 통신

AES-GCM 대칭키, RSA 비대칭키를 통한 안정적이고 빠른 속도의 통신 암호화

PROBLEM

문제 상황

- Web 게임 : 개발자 도구로 통신 내역 손쉬운 노출
- 주요 데이터의 주기적 갱신 과정에서 통신 데이터 보호가 최우선 사항

SOLUTION

하이브리드 암호화 통신

- 주기적 통신·데이터 양 고려, 암호화 속도 보장 필요
- AES-GCM 대칭키 → 세션키로 데이터 암호화
- RSA 비대칭키 → 세션키 교환에 활용

```
public static (string ivBase64, string dataBase64)
Encrypt(string plaintext, byte[] key) // AES-GCM 암호화
{
    byte[] iv = new byte[12];
    new SecureRandom().NextBytes(iv);
    // 매 요청마다 새 IV 생성

    var cipher = new GcmBlockCipher(new AesEngine());
    cipher.Init(true,
        new AeadParameters(
            new KeyParameter(key), 128, iv));

    byte[] plaintextBytes =
        System.Text.Encoding.UTF8.GetBytes(plaintext);
    byte[] output =
        new byte[cipher.GetOutputSize(plaintextBytes.Length)];
    int len = cipher.ProcessBytes(
        plaintextBytes, 0, plaintextBytes.Length, output, 0);
    cipher.DoFinal(output, len);

    return (Convert.ToBase64String(iv),
        Convert.ToBase64String(output));
}
```

start

safe

safe

ping

	Headers	Payload	Preview	Response	Initiator	Timing
1	{			"iv": "R6DCQN1e2H5L3J1D",		
-				"data": "Tkft9Jffo2tArI+NRyMc0yMQoC9RdznLE9I+yy9soR31CDVvNI1A		
-						
-	}					